FEDERATED UNLEARNING THESIS

Month 1 Experiment Report

Centralized ML and Deep Learning Foundations

Period	Month 1 - Weeks 1 to 4
Scope	Python, experiment design, PyTorch, MLP, and CNN foundations
Datasets	MNIST and CIFAR-10
Environment	Python 3.10.20, PyTorch 2.12.1 CPU, fixed seed 42
Prepared	17 August 2026

Month 1 outcome. The project progressed from a 92.19% MNIST linear baseline to a 96.80% MLP and a 98.60% CNN, then established a 71.38% centralized CNN reference on the harder CIFAR-10 task.

This report is written for a beginning AI student. It explains not only what ran, but why each experiment exists, how the measurements were kept reproducible, and what the results do and do not prove.

1. Executive Overview

The first month deliberately stays in centralized learning. Federated Learning and Federated Unlearning depend on the same training loop, loss, evaluation, and image-model concepts. Building those foundations first reduces the risk of mistaking a basic model bug for a federated-learning problem.

92.19% MNIST logistic regression	96.80% MNIST MLP
98.60% MNIST CNN	71.38% CIFAR-10 CNN

Learning progression

Week	Model	Dataset	Accuracy	Purpose
1	Logistic regression	MNIST	92.19%	Establish a simple linear floor.
2	128-unit MLP	MNIST	96.80%	Learn forward pass, loss, backpropagation, optimizer, and learning rate.
3	Small CNN	MNIST	98.60%	Show why local image structure and learned filters help.
3	Small CNN	CIFAR-10	71.38%	Create a harder color-image baseline for later comparison.

Main interpretations

- The MLP's 4.61 percentage-point gain over logistic regression shows that a hidden nonlinear layer learns useful structure beyond one linear decision boundary.
- The MNIST CNN adds another 1.80 points, supporting the idea that preserving local pixel neighborhoods helps recognize handwritten shapes.
- CIFAR-10 is not a failed MNIST result. It is a more difficult task with color, texture, backgrounds, and viewpoint variation, so its baseline must be interpreted separately.
- All reported test numbers come from held-out official test sets after validation-based checkpoint selection.

> Scope discipline. No FedAvg, FedProx, Non-IID partitioning, or unlearning algorithm was implemented in Month 1. Those remain gated behind verified centralized foundations.

2. Experimental Methodology

Every run follows one repeatable lifecycle. A saved JSON configuration is loaded first; the random seed and data split are fixed; the model trains only on training data; validation selects the checkpoint; the test set is evaluated once; and the script automatically writes metrics and visual evidence.

1. Configure	2. Split data	3. Train	4. Validate	5. Test and log
JSON settings and seed	Train / validation / test	Forward, loss, backward, update	Choose best checkpoint	Metrics, plots, timing, config

Data protocol

Experiment	Train	Validation	Test	Augmentation
Week 1 MNIST	49,000	10,500	10,500	None
Week 2 MNIST	51,000	9,000	10,000	None
Week 3 MNIST CNN	51,000	9,000	10,000	None
Week 3 CIFAR-10 CNN	45,000	5,000	10,000	Random crop and horizontal flip (training only)

Metrics in plain language

Metric	Meaning
Accuracy	Fraction of all test examples assigned the correct class.
Precision	Of examples predicted as a class, how many actually belong to it.
Recall	Of examples that truly belong to a class, how many the model found.
Macro F1	Harmonic balance of precision and recall, computed per class and averaged equally across classes.
Confusion matrix	A class-by-class count showing exactly which labels are confused.

Reproducibility controls

- Seed 42 controls data splitting, initial weights, batch order, and augmentation randomness.
- Every run reads a versioned JSON config and writes the required thesis fields automatically.
- The best validation checkpoint is saved before final test evaluation.
- Raw datasets and run outputs remain gitignored; source code and configs are versioned.
- Centralized-only fields are explicit: one client, zero communication rounds, no target client, zero unlearning time, and zero communication cost.

> A fixed seed does not make one result universally true. It makes the run repeatable. Later benchmark stages must use multiple recorded seeds to estimate variability.

3. From a Linear Baseline to a Neural Network

Week 1 - Logistic regression

Logistic regression treats each 28 x 28 image as 784 input values and learns linear class boundaries. It is intentionally simple: a later model should beat it for a clear reason, not merely report an isolated percentage. The config-driven rerun achieved 92.19% accuracy and 92.08% macro F1.

Week 2 - Multilayer perceptron

The MLP adds a 128-unit hidden layer and ReLU activation. For each batch, forward propagation produces logits, cross-entropy measures the error, backpropagation calculates gradients, and Adam updates 101,770 trainable parameters using learning rate 0.001.

Model	Validation accuracy	Test accuracy	Macro F1	CPU train time
Logistic regression	91.76%	92.19%	92.08%	29.2 s
MLP (best epoch)	96.43%	96.80%	96.77%	103.2 s

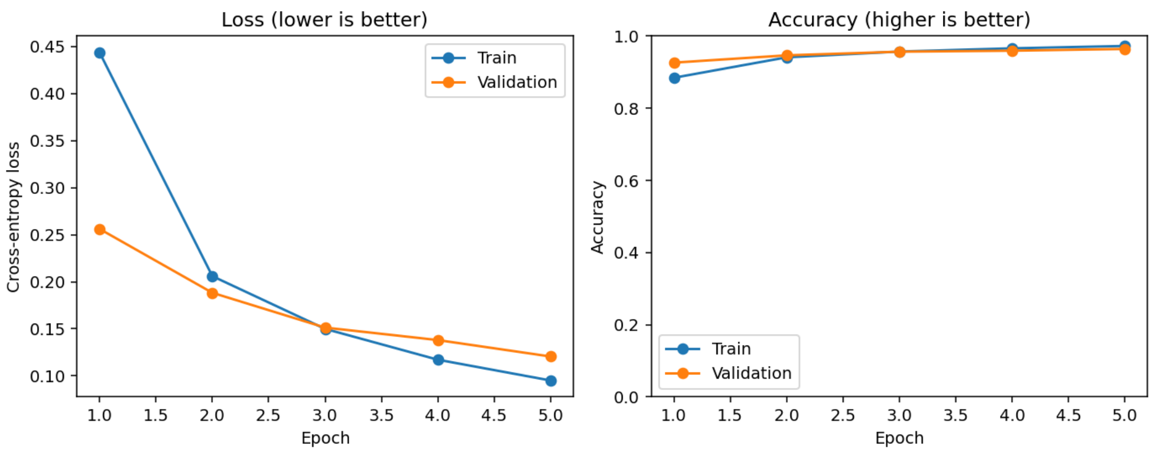


Figure 1. Week 2 MLP learning curves. Both validation accuracy and validation loss improve across 5 epochs.

How to read the curves. Falling loss means the predicted probability distribution is moving closer to the correct labels. Accuracy can stay unchanged for several updates even while loss improves, because loss also reflects prediction confidence.

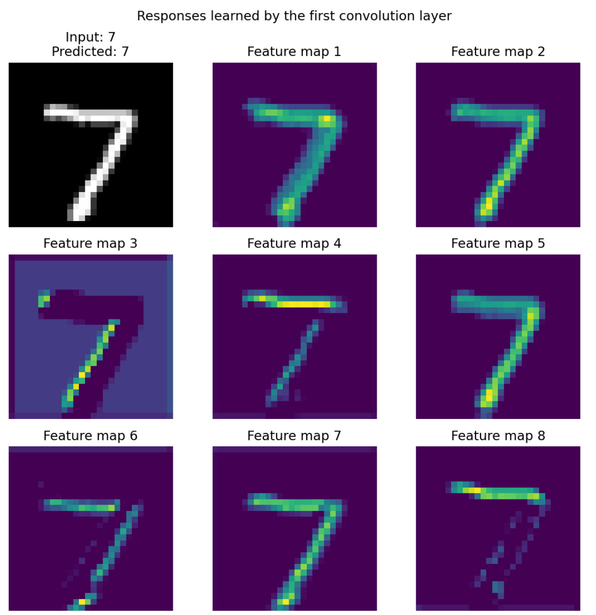
- Train data changes the weights; validation data selects among checkpoints; test data estimates final performance.
- A 0.83-point train-validation gap at epoch 5 does not indicate severe overfitting in this short run.
- The MLP's 96.80% test accuracy is 4.61 points above the Week 1 baseline.

4. MNIST CNN Baseline

Unlike the MLP, a convolutional neural network preserves nearby-pixel structure. A learned 3 x 3 filter slides over the image; its responses form a feature map. Pooling then reduces spatial size while retaining strong responses.

Stage	Output concept	Beginner interpretation
Conv 1 + ReLU	32 feature maps	Learn simple local edges and strokes.
Max pool	Half width and height	Keep strong responses with less computation.
Conv 2 + ReLU	64 feature maps	Combine early patterns into richer shapes.
Adaptive pool	64 x 4 x 4	Create a fixed-size representation.
Classifier	128 hidden units -> 10 logits	Turn learned image features into digit scores.

Parameters	Best epoch	Validation accuracy	Test accuracy	Macro F1	CPU time
151,306	5	98.54%	98.60%	98.60%	6.8 min



Feature-map interpretation

The input's true label is 7 and the prediction is 7. Some filters respond to strong strokes or contours, while others suppress most background pixels. These maps are learned automatically from labels; they were not manually programmed as digit detectors.

Measured improvement

The CNN reaches 98.60% test accuracy: 1.80 points above the MLP and 6.41 points above logistic regression.

Figure 2. Eight first-layer feature maps for one MNIST test image.

The CNN result supports a limited conclusion: local spatial processing helps on this MNIST configuration. It does not prove that every CNN architecture or every dataset will improve by the same amount.

5. CIFAR-10 CNN Baseline

CIFAR-10 contains 32 x 32 color objects under varied backgrounds, positions, textures, and viewpoints. The same two-convolution architecture is used, except the first layer accepts three color channels. Training-only crop and horizontal flip augmentation make the task harder during optimization and improve invariance.

Parameters	Epochs	Best epoch	Validation accuracy	Test accuracy	Macro F1	CPU time
151,882	10	10	72.24%	71.38%	71.07%	17.9 min

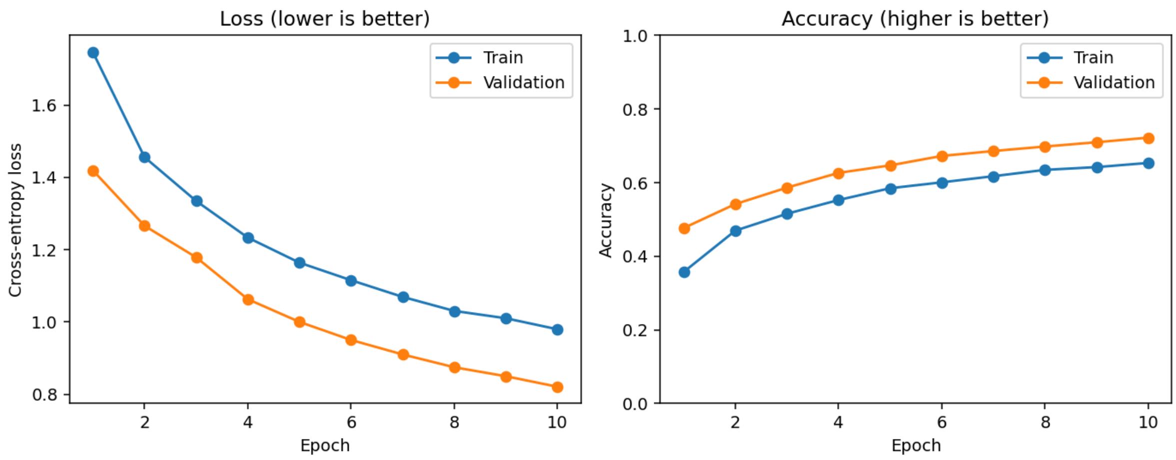


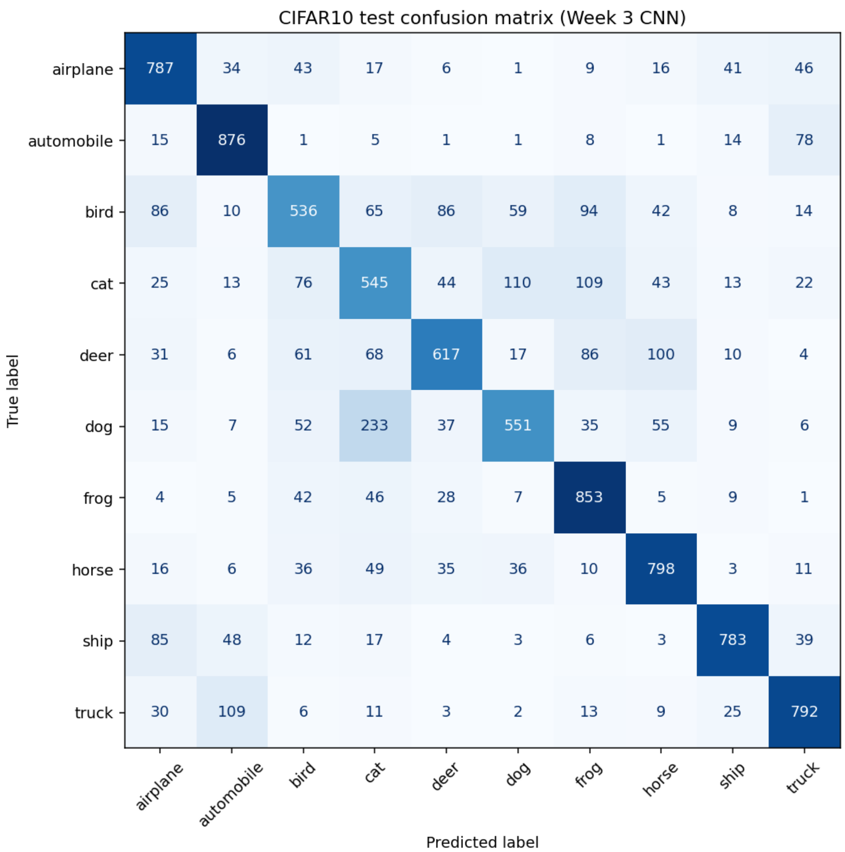
Figure 3. CIFAR-10 learning curves. Validation loss falls and validation accuracy rises throughout all 10 epochs.

Why validation is above training. Training accuracy is measured while random augmentation makes images harder and dropout disables part of the classifier. Both are off during validation. The difference is therefore expected and is not evidence of a data leak.

Class	Accuracy	Class	Accuracy
Airplane	78.7%	Automobile	87.6%
Bird	53.6%	Cat	54.5%
Deer	61.7%	Dog	55.1%
Frog	85.3%	Horse	79.8%
Ship	78.3%	Truck	79.2%

6. Error Analysis and Compute Constraints

One overall percentage can hide systematic weaknesses. The CIFAR-10 confusion matrix shows where the model is reliable and where its learned representation is not yet discriminative enough.



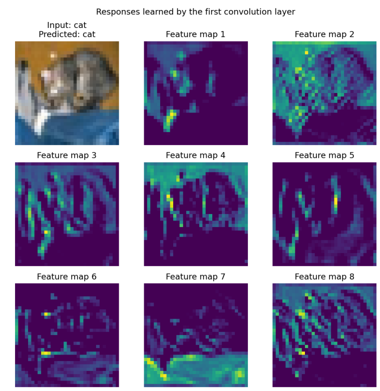


Figure 5. First-layer responses for a test image with true label cat and prediction cat.

Largest visible error: 233 of 1,000 dogs are predicted as cats, while 110 cats are predicted as dogs.

Bird, cat, and dog are the hardest classes (53.6%, 54.5%, and 55.1%). Automobile and frog are the easiest (87.6% and 85.3%).

Figure 4. CIFAR-10 test confusion matrix. Rows are true labels and columns are predictions.

What this means for later FL experiments

- Per-class metrics must remain visible under IID and Non-IID partitions; overall accuracy alone could hide a client dominated by difficult animal classes.
- The 17.9-minute centralized CIFAR run warns against multiplying clients, rounds, and local epochs before MNIST FL is stable.
- The current model is a baseline, not an optimized CIFAR architecture. Honest limitations are more useful than silently increasing model size or compute.

> Integrity note. The Toronto download stalled before training. The replacement Zenodo archive was accepted only after matching TorchVision's exact size (170,498,071 bytes) and MD5 (c58f30108f718f92721af3b95e74349a).

7. SISA Concept, Conclusions, and Gate Readiness

Week 4 reads only the motivation and concept of Bourtoule et al.'s SISA framework. No unlearning method is implemented at this stage. The detailed literature note uses the plan's fixed six-question format.

Question	Concise answer
Problem	Remove a training point's influence without paying full retraining cost.
Gap	Full retraining is strong but expensive; earlier approaches were limited for adaptive/stateful deep learning.
Idea	Shard data, train isolated models, slice incremental training with checkpoints, and aggregate predictions.
Assumption	SISA is designed before training; shards are disjoint; checkpoints and aggregation are available.
Evaluation	Purchase, MNIST, SVHN, ImageNet/Mini-ImageNet, and CIFAR-100 transfer; accuracy and unlearning speed.
Limitation	More shards can reduce accuracy, checkpoints cost storage, and SISA is centralized point-level unlearning rather than client-level FU.

Month 1 conclusions

- The project can train and evaluate PyTorch models with explicit forward, loss, backward, and optimizer steps.
- A small CNN is verified on both required centralized datasets: 98.60% MNIST and 71.38% CIFAR-10 accuracy.
- Accuracy, precision, recall, macro F1, confusion matrices, seeds, checkpoints, configs, and timings are captured automatically.
- The Month 1 code and results support progression to FL concepts only after the beginner Gate 1 explanation check is passed.

Known limitations

All neural-network results use one random seed and one CPU machine; they establish working baselines, not confidence intervals. CIFAR-10 uses a deliberately small model and ten epochs. The report contains no claim about FL convergence, Non-IID behavior, or forgetting because none of those stages has begun.

> Gate 1 status: passed on 16 August 2026. The student's ten-question self-check (reports/gate1_self_check.md) was reviewed and judged to demonstrate the training loop, loss behavior, CNN feature maps, validation/test separation, and the observed accuracy changes in the student's own words. README.md and PROGRESS.md carry the dated closure record; see those files for the current milestone.

References and project evidence

[1] Thesis roadmap: Ke_hoach_6_thang_Thac_si_Federated_Unlearning.md.
[2] Bourtoule et al. (2021), Machine Unlearning, IEEE Symposium on Security and Privacy, DOI: 10.1109/SP40001.2021.00019; arXiv:1912.03817.
[3] Saved configs: configs/month1_week1_mnist_logistic_regression.json, configs/month1_week2_mnist_mlp.json, and both configs/month1_week3_*_cnn.json files.
[4] Measured outputs: results/month1_week1, month1_week2, month1_week3_mnist_cnn, and month1_week3_cifar10_cnn.